

BM3DORNL: High-Performance BM3D Denoising for Neutron Tomography

Chen Zhang¹, Jean-Christophe Bilheux², Dmitry Ganyushin¹, and Pete Peterson¹

¹ Computing and Computational Sciences Directorate, Oak Ridge National Laboratory, Oak Ridge, TN, USA ² Neutron Sciences Directorate, Oak Ridge National Laboratory, Oak Ridge, TN, USA ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Open Journals](#)

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Notice: This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<https://www.energy.gov/doe-public-access-plan>).

Summary

BM3DORNL is a high-performance Python library for denoising neutron and X-ray tomography data using a modified Block-Matching and 3D Filtering (BM3D) algorithm. The library provides two denoising modes: a generic mode for standard noise removal, and a specialized streak mode optimized for removing vertical streak patterns in sinograms that manifest as ring artifacts in reconstructed images. Built with a Rust backend and Python bindings via PyO3, BM3DORNL achieves over 500× faster processing than comparable BM3D implementations while maintaining cross-platform compatibility including native Apple Silicon support.

Statement of Need

Ring artifacts are a persistent challenge in neutron and X-ray computed tomography, arising from variations in detector pixel response, beam intensity fluctuations, and systematic errors (Münch et al., 2009). These artifacts appear as concentric rings in reconstructed images, obscuring sample structure and degrading quantitative analysis. Pre-reconstruction streak removal addresses these artifacts directly in sinograms, avoiding the spatial spreading that occurs during reconstruction. Existing approaches include wavelet-Fourier filtering (Münch et al., 2009), polynomial fitting (Vo et al., 2018), and sorting-based methods (Miqueles et al., 2014), with implementations available in TomoPy (Gürsoy et al., 2014).

Mäkinen et al. (Mäkinen et al., 2021) demonstrated that applying BM3D (Dabov et al., 2007) across multiple scales achieves superior artifact removal by exploiting the self-similarity of streak patterns. However, their bm3d-streak-removal implementation is closed-source, restricted to non-commercial use, and provides only x86_64 binaries incompatible with Apple Silicon and modern Python versions. BM3DORNL provides the neutron imaging community with an open-source, MIT-licensed, high-performance implementation that enables both high-throughput batch processing and interactive parameter tuning.

State of the Field

Several software packages implement pre-processing streak removal for tomography: TomoPy (Gürsoy et al., 2014) provides wavelet-Fourier filtering (Münch et al., 2009) and Vo's sorting/fitting methods (Vo et al., 2018); ASTRA Toolbox (Aarle et al., 2016) offers GPU-accelerated preprocessing; and bm3d-streak-removal (Mäkinen et al., 2021) implements multiscale BM3D but remains closed-source and platform-limited.

Figure 1 shows the benchmark input data: a synthetic sinogram (720×725 pixels) with simulated ring artifacts and the corresponding clean ground truth. All benchmark results are from Linux x86_64 to enable comparison with bm3d-streak-removal, which provides only x86_64 binaries. We compared eight methods: four BM3DORN variants (streak, generic, multiscale, and Fourier-SVD), three TomoPy algorithms (wavelet-Fourier, sorting-fitting, and sorting-based), and the original bm3d-streak-removal. The multiscale variant implements the pyramid approach from Mäkinen et al. (Mäkinen et al., 2021), while Fourier-SVD is a lightweight FFT-guided method that achieves comparable quality at $50\times$ faster speeds.

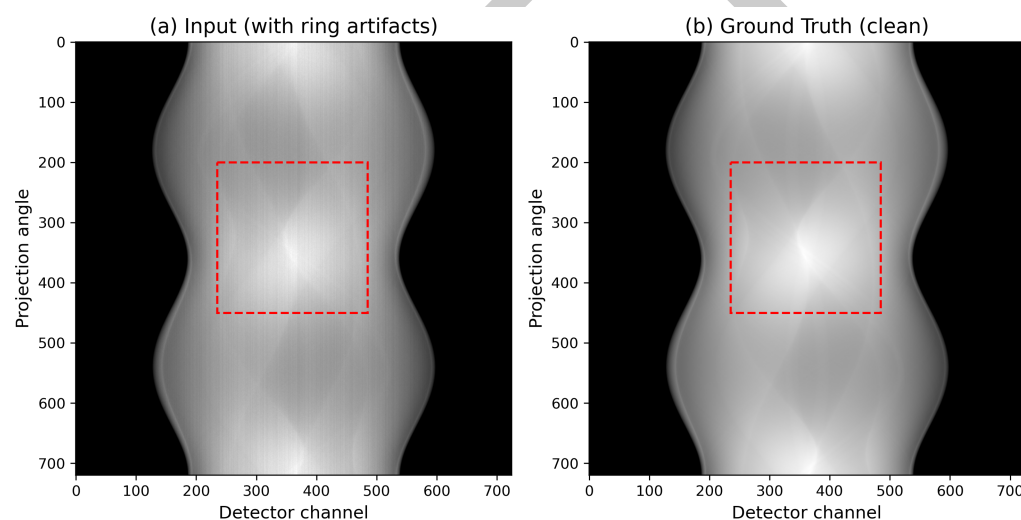


Figure 1: Benchmark input data. (a) Input sinogram with simulated ring artifacts. (b) Ground truth (clean sinogram). The dashed rectangle indicates the crop region shown in Figure 2.

Speed comparison: Figure 3(a) shows processing times ($n=100$ runs on Linux x86_64). Fourier-SVD is the fastest method at 0.017 seconds, achieving $1900\times$ speedup over bm3d-streak-removal (31.8 seconds). The BM3D-based methods span a performance range: standard BM3DORN processes sinograms in 0.094 seconds ($340\times$ faster than bm3d-streak-removal), while multiscale BM3DORN takes 0.389 seconds ($82\times$ faster) due to pyramid processing. TomoPy methods achieve processing times of 0.22–0.29 seconds. Cross-platform performance differs substantially: TomoPy runs $2.5\text{--}3\times$ slower on Apple Silicon than Linux x86_64 (likely due to unoptimized C extensions), whereas BM3DORN runs at comparable speeds (within 10%) across platforms due to its Rust backend.

Quality analysis: Figure 2 shows cropped results from all eight methods with their difference images (result minus ground truth). The benchmark reveals that quality metrics alone do not tell the full story. TomoPy SF achieves the highest SSIM (0.987, see Figure 3(b)), but difference image analysis shows residual vertical stripes in its output. This occurs because SSIM uses local windows (7×7 or 11×11 pixels), and narrow vertical stripes affect few pixels per window. Notably, all BM3DORN variants achieve SSIM scores of 0.943–0.976, with multiscale BM3DORN reaching 0.976 – above the reference bm3d-streak-removal implementation (0.967). Fourier-SVD accomplishes this at 0.017 seconds, demonstrating that aggressive artifact removal

and high-quality reconstruction are achievable without computationally expensive multi-scale processing. The difference images reveal that all BM3DORN variants produce clean vertical patterns, indicating successful artifact removal, while TomoPy methods leave faint residual artifacts.

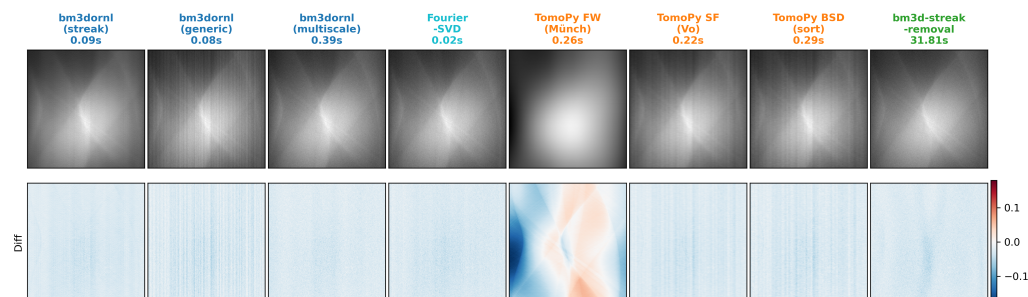


Figure 2: Method comparison. Top row: Cropped results from each method with processing times. Bottom row: Difference images (result minus ground truth) using a zero-centered RdBu colormap—blue indicates artifact removal, red indicates added artifacts. Data from Linux x86_64.

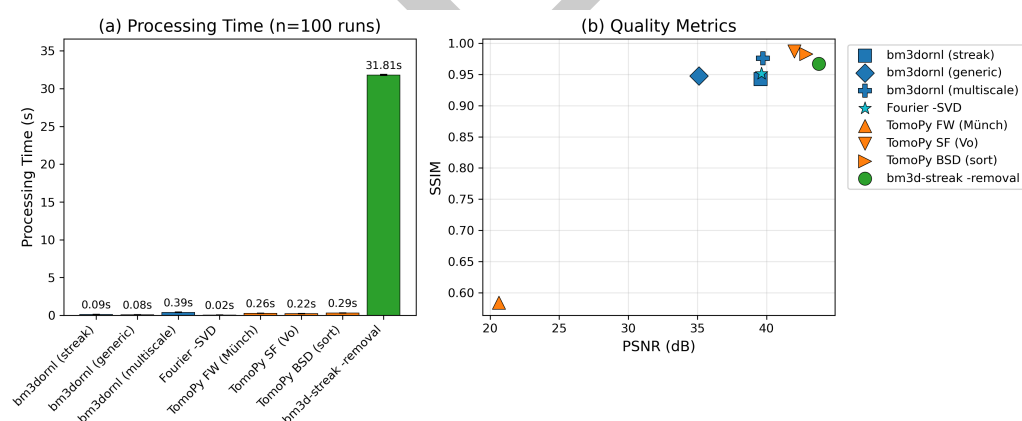


Figure 3: Performance metrics (Linux x86_64, n=100 runs). (a) Processing time comparison on linear scale, showing Fourier-SVD's 1900 $\times$ speedup and standard BM3DORN's 340 $\times$ speedup over bm3d-streak-removal. (b) Quality metrics (PSNR vs SSIM) showing trade-offs between methods. BM3DORN variants (blue/cyan), TomoPy (orange), bm3d-streak-removal (green).

Platform support: bm3d-streak-removal is unavailable on Apple Silicon (x86_64 binaries only), incompatible with Python 3.11+, and restricted to non-commercial use. BM3DORN provides native performance on all platforms, supports Python 3.12+, and uses the MIT license for unrestricted commercial use.

Software Design

BM3DORN employs a hybrid Python-Rust architecture: the core algorithm is implemented in Rust using the rayon crate for work-stealing parallelism, with Python bindings via PyO3 for seamless NumPy integration.

Key optimizations:

- **Integral image pre-screening:** Before computing expensive patch distances, the block matching stage uses integral images to compute mean and norm bounds in $O(1)$ time.

- Patches that fail these bounds are skipped, eliminating approximately 80% of distance calculations.
- **FFT with SIMD acceleration:** BM3DORNL defaults to FFT-based transforms with batched row/column passes and specialized 8×8 fast paths. FFT plans are computed once and shared across threads via Arc, and a direct-mapped transform cache eliminates redundant computations. An alternative Walsh-Hadamard path is available for multiplication-free processing.
 - **SIMD-optimized block matching:** The 8×8 patch distance computation uses platform-specific SIMD instructions with a contiguous-memory fast path, reducing the block matching inner loop to near-hardware-limit throughput.
 - **Tile-owned aggregation:** Sinograms are processed in tiles that fit in L2/L3 cache, with SIMD-optimized weight accumulation. Per-worker scratch buffers are reused across kernel runs, minimizing memory allocation overhead.

The library provides a minimal Python API:

```
from bm3dornl import bm3d_ring_artifact_removal

# "streak" mode is the default; sigma_random=0.0 enables
# automatic noise estimation (default is 0.1)
cleaned = bm3d_ring_artifact_removal(sinogram, sigma_random=0.0)
```

GUI application: BM3DORNL includes a native GUI built with the egui framework for Rust, installable via `pip install bm3dornl[gui]` or Homebrew on macOS. The GUI enables interactive parameter tuning, side-by-side comparison with difference visualization, HDF5/TIFF file loading with dataset browsing, and real-time processing feedback with a fast-mode toggle. At 0.08 seconds per sinogram, scientists can explore parameter space interactively at over 12 frames per second—something impractical with `bm3d-streak-removal`'s 40-second processing time.

Research Impact

BM3DORNL is being integrated into processing pipelines at the VENUS and MARS beamlines at Oak Ridge National Laboratory. The library provides multiple performance tiers: Fourier-SVD enables real-time parameter exploration at 0.017 seconds per sinogram ($1900\times$ faster than `bm3d-streak-removal`), standard BM3DORNL offers robust denoising at 0.094 seconds ($340\times$ speedup), and multiscale BM3DORNL provides the highest structural fidelity at 0.389 seconds ($82\times$ speedup). For batch processing, a 1000-sinogram dataset that would take 9 hours with `bm3d-streak-removal` completes in 17 seconds (Fourier-SVD), 94 seconds (standard BM3DORNL), or 6.5 minutes (multiscale BM3DORNL).

The hybrid Rust-Python architecture demonstrates a modern approach to scientific software development: Rust provides memory safety and performance portability (single codebase compiles natively to arm64 and x86_64), while Python ensures integration with the NumPy/SciPy ecosystem. This pattern is increasingly valuable as the scientific computing community diversifies hardware platforms.

BM3DORNL fills a critical gap: the neutron imaging community needed an open-source, actively maintained BM3D implementation that works on modern platforms. The MIT license enables unrestricted use at national facilities, and native Apple Silicon support ensures scientists can use contemporary hardware for analysis workstations.

AI Usage Disclosure

Generative AI tools (Claude) were used for code assistance and documentation drafting. All AI-generated content was reviewed, tested, and validated by the authors.

Acknowledgements

This research used resources at the Spallation Neutron Source, a DOE Office of Science User Facility operated by Oak Ridge National Laboratory. This work is supported by the U.S. Department of Energy under Contract No. DE-AC05-00OR22725.

References

- Aarle, W. van, Palenstijn, W. J., Cant, J., Janssens, E., Bleichrodt, F., Dabrovolski, A., De Beenhouwer, J., Batenburg, K. J., & Sijbers, J. (2016). Fast and flexible x-ray tomography using the ASTRA toolbox. *Optics Express*, 24(22), 25129–25147. <https://doi.org/10.1364/OE.24.025129>
- Dabov, K., Foi, A., Katkovnik, V., & Egiazarian, K. (2007). Image denoising by sparse 3-d transform-domain collaborative filtering. *IEEE Transactions on Image Processing*, 16(8), 2080–2095. <https://doi.org/10.1109/TIP.2007.901238>
- Gürsoy, D., De Carlo, F., Xiao, X., & Jacobsen, C. (2014). TomoPy: A framework for the analysis of synchrotron tomographic data. *Journal of Synchrotron Radiation*, 21(5), 1188–1193. <https://doi.org/10.1107/S1600577514013939>
- Mäkinen, Y., Marchesini, S., & Foi, A. (2021). Ring artifact reduction via multiscale nonlocal collaborative filtering of spatially correlated noise. *Journal of Synchrotron Radiation*, 28(3), 876–888. <https://doi.org/10.1107/S1600577521001910>
- Miqueles, E. X., Rinkel, J., O'Dowd, F., & Bermúdez, J. S. V. (2014). Generalized titarenko's algorithm for ring artefacts reduction. *Journal of Synchrotron Radiation*, 21(6), 1333–1346. <https://doi.org/10.1107/S1600577514016919>
- Münch, B., Trtik, P., Marone, F., & Stampanoni, M. (2009). Stripe and ring artifact removal with combined wavelet—fourier filtering. *Optics Express*, 17(10), 8567–8591. <https://doi.org/10.1364/OE.17.008567>
- Vo, N. T., Atwood, R. C., & Drakopoulos, M. (2018). Superior techniques for eliminating ring artifacts in x-ray micro-tomography. *Optics Express*, 26(22), 28396–28412. <https://doi.org/10.1364/OE.26.028396>